

▼ Dota 2 Pro Matches — Análisis Exploratorio

Dataset: `tb_pro_players_matches.csv`

Partidas: 47 150 partidas profesionales (2019–2021)

Columnas: 317 (3 identificadores + 72 estadísticas históricas + 242 pick rates de héroes)

¿Qué representa cada fila?

Cada fila es **1 partida real** (`match_id` único). Los valores numéricos son el **promedio histórico** del equipo en las últimas N partidas **antes** de ese match.

No son estadísticas de lo que ocurrió en esa partida, sino el perfil de forma de cada equipo.

Columna	Descripción
<code>match_id</code>	ID único de la partida
<code>dt_match</code>	Fecha/hora UTC
<code>radiant_win</code>	True = ganó Radiant
<code>*_avg_r / *_avg_d</code>	Estadísticas históricas de Radiant / Dire
<code>hero_<ID>_avg_r/d</code>	Pick rate del héroe en la ventana histórica

2-819 partidas (6%) no tienen héroes registrados. Se excluyen del análisis táctico.

```
# Instalar dependencias si es necesario (Colab)
# !pip install pandas matplotlib seaborn numpy
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'figure.facecolor': '#1a1a2e',
    'axes.facecolor': '#16213e',
    'axes.edgecolor': '#444',
    'axes.labelcolor': '#ddd',
    'xtick.color': '#bbb',
    'ytick.color': '#bbb',
    'text.color': '#eee',
    'grid.color': '#ffffff18',
    'grid.linestyle': '--',
    'axes.titlesize': 13,
    'axes.titleweight': 'bold',
})

GOLD = '#c89b3c'
GREEN = '#3cb56a'
RED = '#e05c5c'
BLUE = '#4a90d9'
PURPLE = '#9b59b6'
CYAN = '#1abc9c'
```

```
# — Cargar dataset —————
# Si estás en Colab, sube el archivo o monta Google Drive:
# from google.colab import files; files.upload()
# O: from google.colab import drive; drive.mount('/content/drive')

CSV_PATH = 'tb_pro_players_matches.csv' # ajusta la ruta si es necesario
df = pd.read_csv(CSV_PATH)
df['dt_match'] = pd.to_datetime(df['dt_match'], errors='coerce')
df['_year'] = df['dt_match'].dt.year

# Helper para identificar columnas de pick-rate de héroes
def es_col_heroe(col):
    partes = col.split('_')
    if len(partes) < 4: return False
    try: int(partes[1]); return True
    except ValueError: return False

hero_cols_r = [c for c in df.columns if c.startswith('hero_') and c.endswith('_avg_r') and es_col_heroe(c)]
hero_cols_d = [c for c in df.columns if c.startswith('hero_') and c.endswith('_avg_d') and es_col_heroe(c)]

df['_has_heroes_r'] = (df[hero_cols_r].fillna(0) > 0).any(axis=1)
```

```

df['_has_heroes_d'] = (df[hero_cols_d].fillna(0) > 0).any(axis=1)
df['_has_heroes_r'] = df['_has_heroes_r'] & df['_has_heroes_d']

df_valid = df[df['_has_heroes']].copy()
df_invalid = df[~df['_has_heroes']].copy()

print(f'Total partidas : {len(df):,}')
print(f'Con héroes : {len(df_valid):,} ({len(df_valid)/len(df)*100:.1f}%)')
print(f'Sin héroes : {len(df_invalid):,} ({len(df_invalid)/len(df)*100:.1f}%)')
print(f'Radiant gana : {df_valid["radiant_win"].mean()*100:.1f}%)')

```

```

Total partidas : 47,150
Con héroes : 42,932 (91.1%)
Sin héroes : 4,218 (8.9%)
Radiant gana : 52.1%

```

1 — Distribución de partidas válidas e inválidas por año

Las partidas "sin héroes" son filas que tienen `match_id` y resultado pero **cero columnas de pick rate con valor > 0**. Corresponden a equipos sin historial previo registrado en OpenDota (debuts, datos faltantes en la API).

```

by_year = df.groupby('_year').agg(
    validas=('_has_heroes', 'sum'),
    total=('_has_heroes', 'count')
).reset_index()
by_year['invalidas'] = by_year['total'] - by_year['validas']

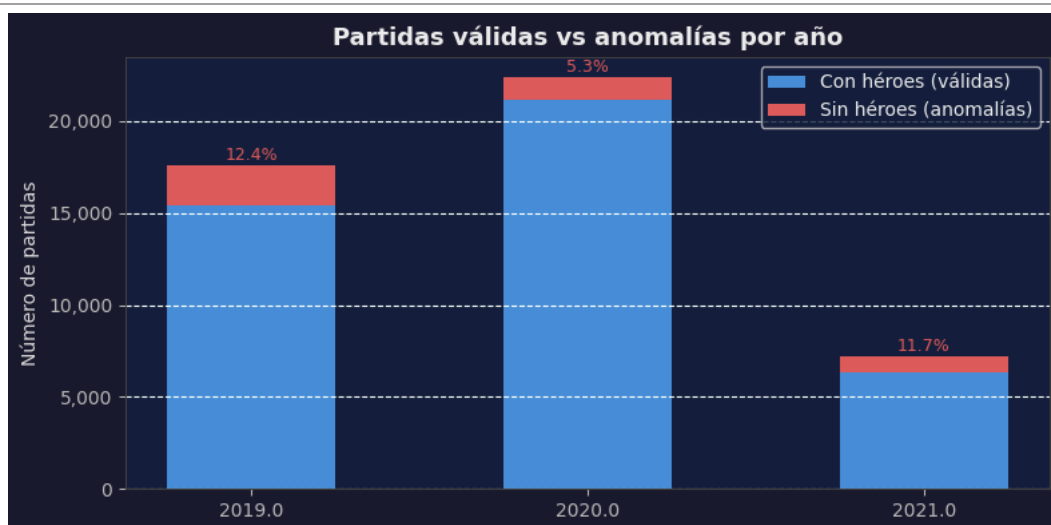
fig, ax = plt.subplots(figsize=(8, 4))
years = by_year['_year'].astype(str)
x = np.arange(len(years))
w = 0.5

bars_v = ax.bar(x, by_year['validas'], width=w, color=BLUE, label='Con héroes (válidas)')
bars_i = ax.bar(x, by_year['invalidas'], width=w, bottom=by_year['validas'], color=RED, label='Sin héroes (anomalías)')

for bar, inv, val in zip(bars_i, by_year['invalidas'], by_year['validas']):
    pct = inv / (inv + val) * 100
    ax.text(bar.get_x() + bar.get_width()/2, val + inv + 100,
            f'{pct:.1f}%', ha='center', va='bottom', fontsize=9, color=RED)

ax.set_xticks(x)
ax.set_xticklabels(years)
ax.set_title('Partidas válidas vs anomalías por año')
ax.set_ylabel('Número de partidas')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda v, _: f'{int(v):,}'))
ax.legend()
ax.grid(axis='y')
plt.tight_layout()
plt.show()

```



2 — Win Rate global: Radiant vs Dire

Radiant tiene una ligera ventaja estructural en Dota 2, conocida y atribuida a la geometría del mapa (la base de Radiant está en la esquina inferior izquierda, con acceso más fácil a Roshan).

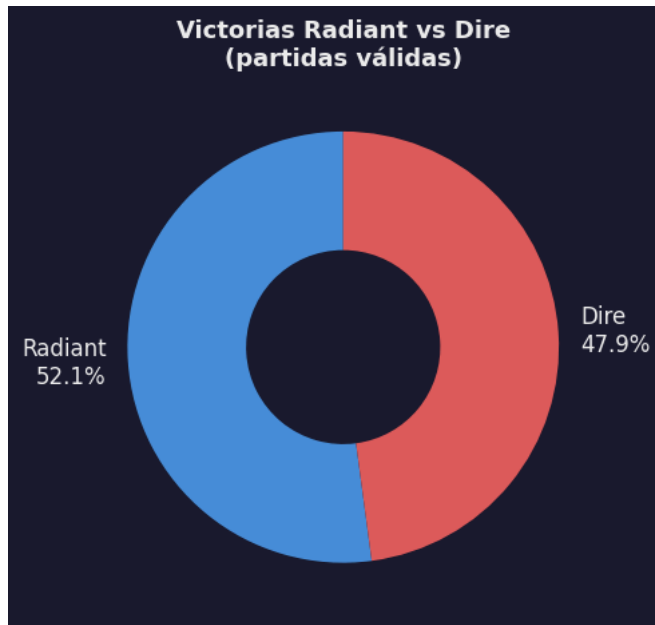
```
r_wins = df_valid['radiant_win'].sum()
```

```

d_wins = len(df_valid) - r_wins
sizes = [r_wins, d_wins]
labels = [f'Radiant\n{r_wins/len(df_valid)*100:.1f}%', f'Dire\n{d_wins/len(df_valid)*100:.1f}%']
colors = [BLUE, RED]

fig, ax = plt.subplots(figsize=(5, 5))
wedges, texts = ax.pie(sizes, labels=labels, colors=colors, startangle=90,
                        wedgeprops=dict(width=0.55), textprops=dict(fontsize=12))
ax.set_title('Victorias Radiant vs Dire\n(partidas válidas)')
plt.tight_layout()
plt.show()

```



3 — GPM y XPM: Radiant vs Dire

GPM (Gold Per Minute) y **XPM** (Experience Per Minute) son los dos indicadores primarios de eficiencia económica. Se comparan los promedios globales de ambos equipos.

```

metrics = {
    'GPM (oro/min)': ('gold_per_min_avg_r', 'gold_per_min_avg_d'),
    'XPM (XP/min)': ('xp_per_min_avg_r', 'xp_per_min_avg_d'),
}

labels_m = list(metrics.keys())
vals_r = [df_valid[v[0]].mean() for v in metrics.values()]
vals_d = [df_valid[v[1]].mean() for v in metrics.values()]

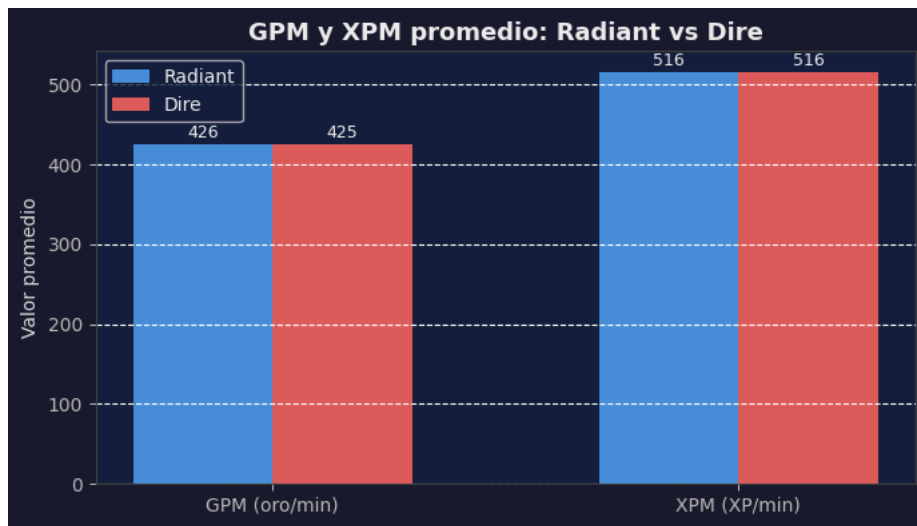
x = np.arange(len(labels_m))
w = 0.3

fig, ax = plt.subplots(figsize=(7, 4))
br = ax.bar(x - w/2, vals_r, width=w, color=BLUE, label='Radiant')
bd = ax.bar(x + w/2, vals_d, width=w, color=RED, label='Dire')

for bar in list(br) + list(bd):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 3,
            f'{bar.get_height():.0f}', ha='center', va='bottom', fontsize=9)

ax.set_xticks(x)
ax.set_xticklabels(labels_m)
ax.set_title('GPM y XPM promedio: Radiant vs Dire')
ax.set_ylabel('Valor promedio')
ax.legend()
ax.grid(axis='y')
plt.tight_layout()
plt.show()

```



4 — Win Rate de los Top 50 héroes más elegidos

Se calcula el **win rate promedio** de cada héroe: promedio de `radiant_win` en las partidas donde ese héroe tiene pick rate > 0 en Radiant, vs `1 - radiant_win` en las de Dire. Las barras se colorean: **verde** > 55%, **rojo** < 45%, **dorado** = rango medio.

```
# Cargar nombres de héroes desde la API de OpenDota
import urllib.request, json as json_lib

hero_names = {}
try:
    url = 'https://api.opendota.com/api/heroes'
    with urllib.request.urlopen(url, timeout=5) as resp:
        heroes_api = json_lib.loads(resp.read())
        hero_names = {h['id']: h['localized_name'] for h in heroes_api}
except Exception:
    print('No se pudo cargar la API — se usarán IDs como nombres.')

# Calcular win rate por héroe — misma lógica que el backend /api/graficos
# freq = SUMA de pick rates (float), NO conteo de filas.
# Esto garantiza que el ranking de héroes sea idéntico al de la web app.
resultados = {}

for col_r in hero_cols_r:
    hid_str = col_r.split('_')[1]
    hid = int(hid_str)
    freq_r = float(df_valid[col_r].fillna(0).sum()) # suma de tasas de pick
    mask_r = df_valid[col_r].fillna(0) > 0
    part_r = df_valid[mask_r]
    wins_r = int(part_r['radiant_win'].sum()) # victorias cuando Radiant usa este héroe
    if hid not in resultados:
        resultados[hid] = {'freq': 0, 'wins': 0, 'total': 0}
    resultados[hid]['freq'] += freq_r
    resultados[hid]['wins'] += wins_r
    resultados[hid]['total'] += len(part_r)

for col_d in hero_cols_d:
    hid_str = col_d.split('_')[1]
    hid = int(hid_str)
    freq_d = float(df_valid[col_d].fillna(0).sum()) # suma de tasas de pick
    mask_d = df_valid[col_d].fillna(0) > 0
    part_d = df_valid[mask_d]
    wins_d = int((~part_d['radiant_win']).sum()) # victorias cuando Dire usa este héroe
    if hid not in resultados:
        resultados[hid] = {'freq': 0, 'wins': 0, 'total': 0}
    resultados[hid]['freq'] += freq_d
    resultados[hid]['wins'] += wins_d
    resultados[hid]['total'] += len(part_d)

hero_stats = []
for hid, datos in resultados.items():
    if datos['total'] < 50: continue
    hero_stats.append({
        'id': hid,
        'name': hero_names.get(hid, f'Hero {hid}'),
        'freq': datos['freq'],
        'wr': datos['wins'] / datos['total']
    })
```

```

})

# Ordenar por freq (suma de pick rates) descendente → top 50 → luego por wr para el gráfico
hdf = pd.DataFrame(hero_stats).sort_values('freq', ascending=False).head(50)
hdf = hdf.sort_values('wr', ascending=False)

def hero_color(wr):
    if wr >= 0.55: return GREEN
    if wr <= 0.45: return RED
    return GOLD

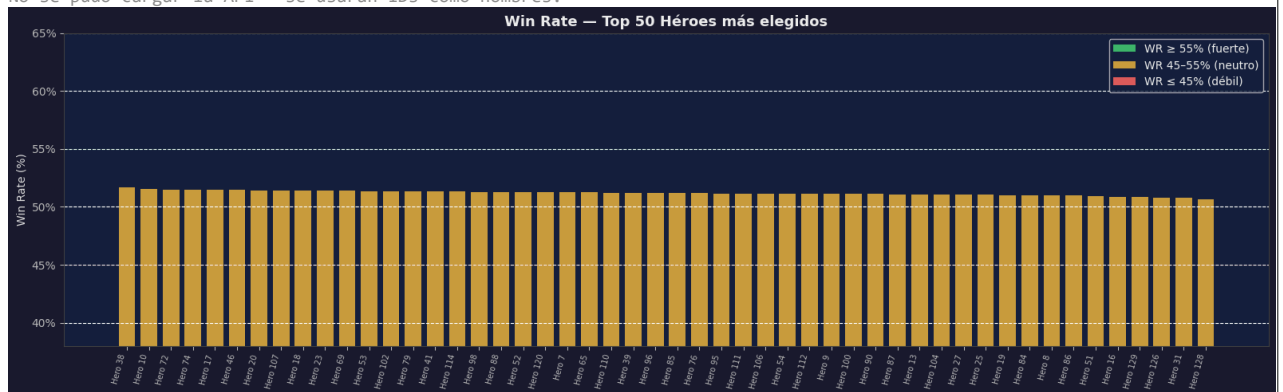
colors_h = [hero_color(w) for w in hdf['wr']]

fig, ax = plt.subplots(figsize=(16, 5))
bars = ax.bar(range(len(hdf)), hdf['wr'] * 100, color=colors_h, width=0.75)
ax.axhline(50, color='white', linewidth=0.8, linestyle='--', alpha=0.5)
ax.axhline(55, color=GREEN, linewidth=0.6, linestyle=':', alpha=0.4)
ax.axhline(45, color=RED, linewidth=0.6, linestyle=':', alpha=0.4)
ax.set_xticks(range(len(hdf)))
ax.set_xticklabels(hdf['name'], rotation=75, ha='right', fontsize=7)
ax.set_ylim(38, 65)
ax.set_ylabel('Win Rate (%)')
ax.set_title('Win Rate — Top 50 Héroes más elegidos')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda v, _: f'{v:.0f}%'))
ax.grid(axis='y')

from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor=GREEN, label='WR ≥ 55% (fuerte)'),
    Patch(facecolor=GOLD, label='WR 45-55% (neutro)'),
    Patch(facecolor=RED, label='WR ≤ 45% (débil)'),
]
ax.legend(handles=legend_elements, loc='upper right')
plt.tight_layout()
plt.show()

```

No se pudo cargar la API — se usarán IDs como nombres.



5 — KDA promedio: ganadores vs perdedores

KDA = (Kills + Assists) / Deaths — indicador sintético de rendimiento de combate.

Se compara el KDA histórico de los equipos que **ganaron** vs los que **perdieron** en cada partida.

```

# Ganadores: Radiant cuando radiant_win=True, Dire cuando radiant_win=False
kda_win = pd.concat([
    df_valid.loc[df_valid['radiant_win'] == True, 'kda_avg_r'],
    df_valid.loc[df_valid['radiant_win'] == False, 'kda_avg_d']
])
kda Lose = pd.concat([
    df_valid.loc[df_valid['radiant_win'] == False, 'kda_avg_r'],
    df_valid.loc[df_valid['radiant_win'] == True, 'kda_avg_d']
])

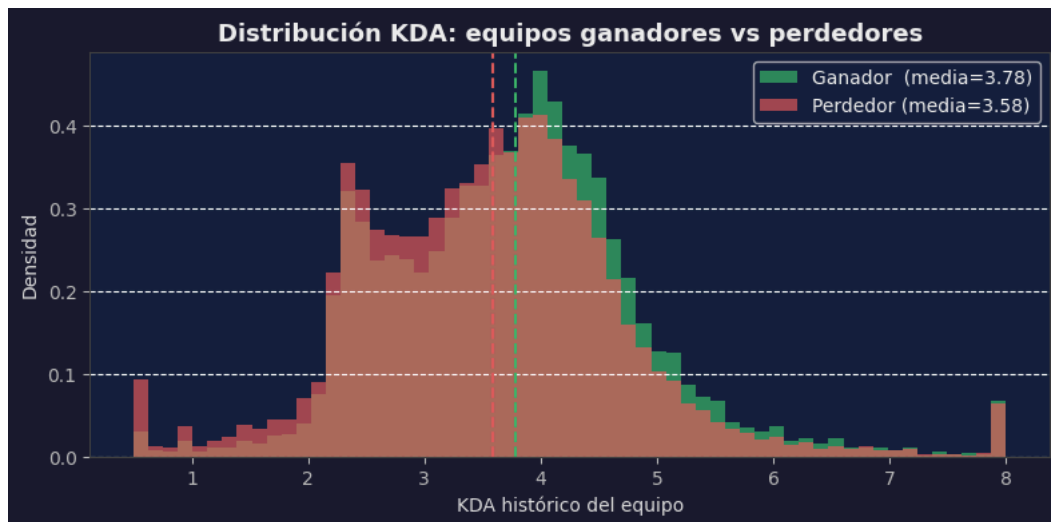
fig, ax = plt.subplots(figsize=(8, 4))

```

```

bins = np.linspace(0.5, 8, 60)
ax.hist(kda_win.clip(0.5, 8), bins=bins, color=GREEN, alpha=0.7, label=f'Ganador (media={kda_win.mean():.2f})', density=
ax.hist(kda_lose.clip(0.5, 8), bins=bins, color=RED, alpha=0.7, label=f'Perdedor (media={kda_lose.mean():.2f})', density=
ax.axvline(kda_win.mean(), color=GREEN, linewidth=1.5, linestyle='--')
ax.axvline(kda_lose.mean(), color=RED, linewidth=1.5, linestyle='--')
ax.set_xlabel('KDA histórico del equipo')
ax.set_ylabel('Densidad')
ax.set_title('Distribución KDA: equipos ganadores vs perdedores')
ax.legend()
ax.grid(axis='y')
plt.tight_layout()
plt.show()

```



6 — Control de visión: wards colocadas y destruidas

La "guerra de visión" es uno de los aspectos más técnicos del juego profesional.

Se comparan wards **observer** (visión ofensiva) y **sentry** (detección) entre ganadores y perdedores.

```

def split_win_lose(col_r, col_d):
    win = pd.concat([df_valid.loc[df_valid['radiant_win']==True, col_r],
                    df_valid.loc[df_valid['radiant_win']==False, col_d]])
    lose = pd.concat([df_valid.loc[df_valid['radiant_win']==False, col_r],
                    df_valid.loc[df_valid['radiant_win']==True, col_d]])
    return win.mean(), lose.mean()

vision_metrics = {
    'Observer\ncolocadas': ('observer_uses_avg_r', 'observer_uses_avg_d'),
    'Observer\ndestruidas': ('observer_kills_avg_r', 'observer_kills_avg_d'),
    'Sentry\ncolocadas': ('sentry_uses_avg_r', 'sentry_uses_avg_d'),
    'Sentry\ndestruidas': ('sentry_kills_avg_r', 'sentry_kills_avg_d'),
}

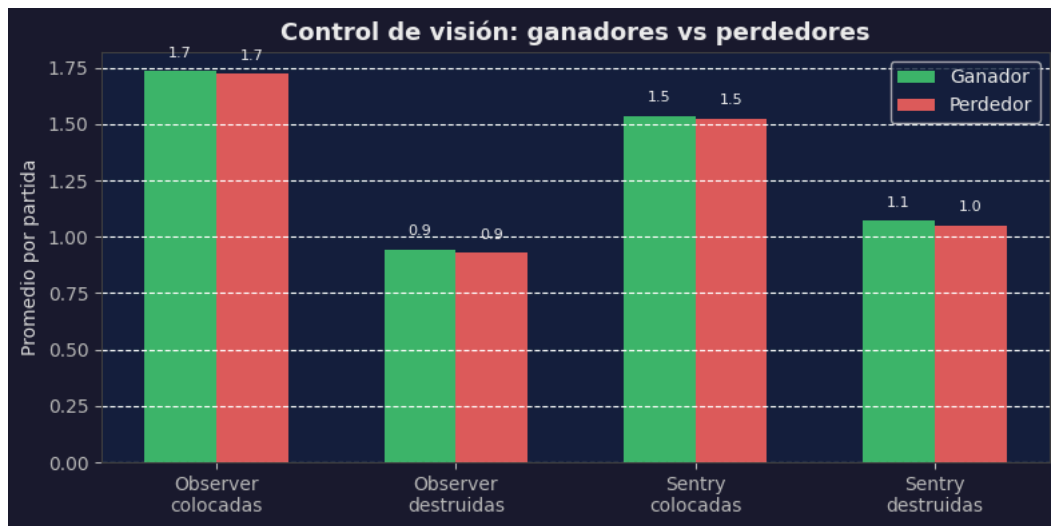
vm_labels = list(vision_metrics.keys())
vm_win = []
vm_lose = []
for cr, cd in vision_metrics.values():
    w, l = split_win_lose(cr, cd)
    vm_win.append(w); vm_lose.append(l)

x = np.arange(len(vm_labels))
w = 0.3
fig, ax = plt.subplots(figsize=(8, 4))
bw = ax.bar(x - w/2, vm_win, width=w, color=GREEN, label='Ganador')
bl = ax.bar(x + w/2, vm_lose, width=w, color=RED, label='Perdedor')

for bar in list(bw) + list(bl):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.05,
            f'{bar.get_height():.1f}', ha='center', va='bottom', fontsize=8)

ax.set_xticks(x)
ax.set_xticklabels(vm_labels)
ax.set_title('Control de visión: ganadores vs perdedores')
ax.set_ylabel('Promedio por partida')
ax.legend()
ax.grid(axis='y')
plt.tight_layout()
plt.show()

```



7 — Torres vs Kills: ¿qué predice mejor la victoria?

Hipótesis: destruir torres es más determinante que acumular kills.

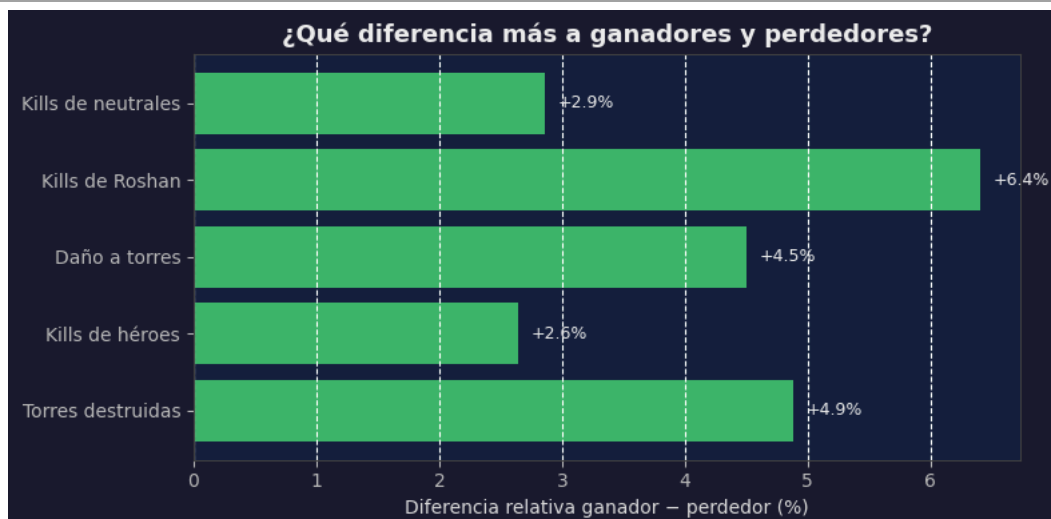
Se comparan las diferencias relativas (ganador – perdedor) para `tower_kills_avg` y `hero_kills_avg`.

```
objectives = {
    'Torres destruidas': ('tower_kills_avg_r', 'tower_kills_avg_d'),
    'Kills de héroes': ('hero_kills_avg_r', 'hero_kills_avg_d'),
    'Daño a torres': ('tower_damage_avg_r', 'tower_damage_avg_d'),
    'Kills de Roshan': ('roshan_kills_avg_r', 'roshan_kills_avg_d'),
    'Kills de neutrales': ('neutral_kills_avg_r', 'neutral_kills_avg_d'),
}

diffs_abs = []
diffs_pct = []
for cr, cd in objectives.values():
    w, l = split_win_lose(cr, cd)
    diffs_abs.append(w - l)
    diffs_pct.append((w - l) / l * 100 if l > 0 else 0)

obj_labels = list(objectives.keys())
colors_obj = [GREEN if d > 0 else RED for d in diffs_pct]

fig, ax = plt.subplots(figsize=(8, 4))
bars = ax.barh(obj_labels, diffs_pct, color=colors_obj)
ax.axvline(0, color='white', linewidth=0.8)
for bar, val in zip(bars, diffs_pct):
    ax.text(bar.get_width() + 0.1, bar.get_y() + bar.get_height()/2,
            f'+{val:.1f}%' if val >= 0 else f'{val:.1f}%',
            va='center', fontsize=9)
ax.set_xlabel('Diferencia relativa ganador – perdedor (%)')
ax.set_title('¿Qué diferencia más a ganadores y perdedores?')
ax.grid(axis='x')
plt.tight_layout()
plt.show()
```



8 — Duración de partidas: ganando vs perdiendo

Los equipos generalmente **ganan en partidas más cortas** que sus derrotas, porque los equipos perdedores alargan el juego buscando una remontada.

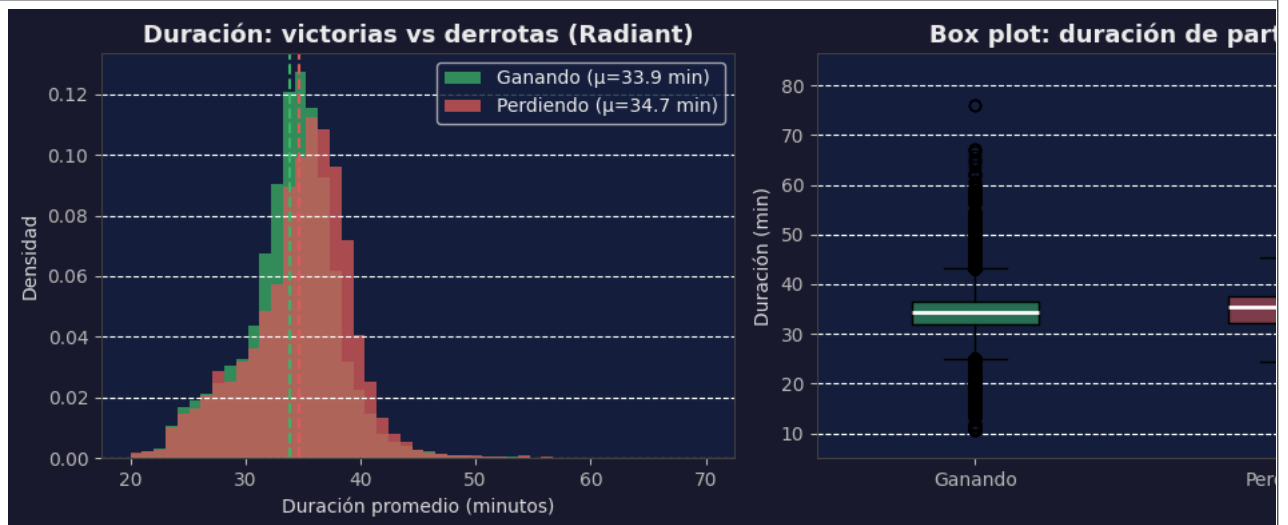
```
dur_win_r = df_valid['duration_avg_win_r'].dropna()
dur Lose_r = df_valid['duration_avg Lose_r'].dropna()

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

# Histograma de duración
bins = np.linspace(20, 70, 50)
axes[0].hist(dur_win_r, bins=bins, color=GREEN, alpha=0.75, label=f'Ganando ( $\mu$ ={{dur_win_r.mean():.1f}} min)', density=True)
axes[0].hist(dur Lose_r, bins=bins, color=RED, alpha=0.75, label=f'Perdiendo ( $\mu$ ={{dur Lose_r.mean():.1f}} min)', density=True)
axes[0].axvline(dur_win_r.mean(), color=GREEN, linewidth=1.5, linestyle='--')
axes[0].axvline(dur Lose_r.mean(), color=RED, linewidth=1.5, linestyle='--')
axes[0].set_xlabel('Duración promedio (minutos)')
axes[0].set_ylabel('Densidad')
axes[0].set_title('Duración: victorias vs derrotas (Radiant)')
axes[0].legend()
axes[0].grid(axis='y')

# Box plot comparativo
data_box = [dur_win_r, dur Lose_r]
bp = axes[1].boxplot(data_box, patch_artist=True, widths=0.4,
                    medianprops=dict(color='white', linewidth=2))
bp['boxes'][0].set_facecolor(GREEN + '88')
bp['boxes'][1].set_facecolor(RED + '88')
axes[1].set_xticklabels(['Ganando', 'Perdiendo'])
axes[1].set_ylabel('Duración (min)')
axes[1].set_title('Box plot: duración de partidas')
axes[1].grid(axis='y')

plt.tight_layout()
plt.show()
```



9 — Economía: GPM de ganadores vs perdedores a lo largo del tiempo

Se analiza si la ventaja económica (GPM) de los ganadores fue consistente en 2019, 2020 y 2021.

```
years_list = sorted(df_valid['_year'].dropna().unique())
gpm_win_yr = []
gpm Lose_yr = []

for yr in years_list:
    sub = df_valid[df_valid['_year'] == yr]
    w, l = split_win Lose('gold_per_min_avg_r', 'gold_per_min_avg_d')
    # Recalcular para el subconjunto del año
    gpm_w = pd.concat([
        sub.loc[sub['radiant_win']==True, 'gold_per_min_avg_r'],
        sub.loc[sub['radiant_win']==False, 'gold_per_min_avg_d']
    ]).mean()
    gpm_l = pd.concat([
        sub.loc[sub['radiant_win']==False, 'gold_per_min_avg_r'],
        sub.loc[sub['radiant_win']==True, 'gold_per_min_avg_d']
    ]).mean()
```

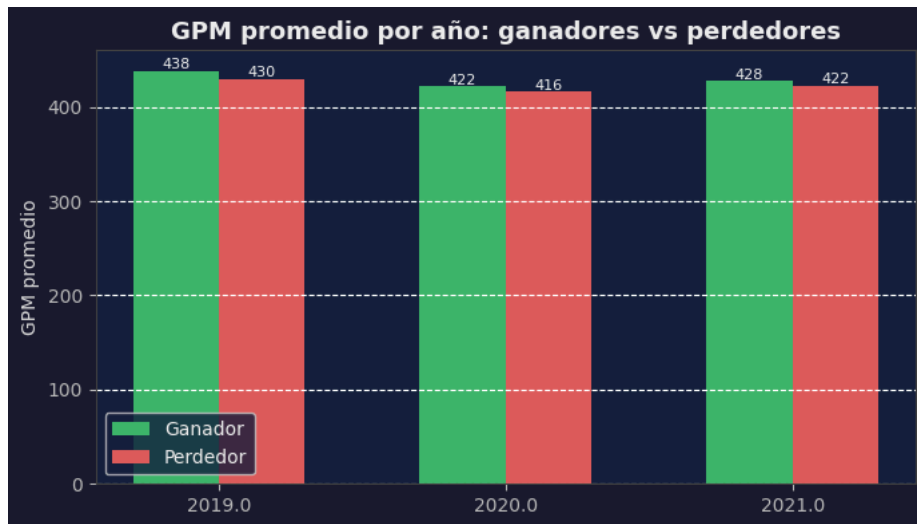


```

    ]).mean()
    gpm_win_yr.append(gpm_w)
    gpm_lose_yr.append(gpm_l)

x = np.arange(len(years_list))
w = 0.3
fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(x - w/2, gpm_win_yr, width=w, color=GREEN, label='Ganador')
ax.bar(x + w/2, gpm_lose_yr, width=w, color=RED, label='Perdedor')
ax.set_xticks(x)
ax.set_xticklabels([str(y) for y in years_list])
ax.set_ylabel('GPM promedio')
ax.set_title('GPM promedio por año: ganadores vs perdedores')
ax.legend()
ax.grid(axis='y')
for i, (wv, lv) in enumerate(zip(gpm_win_yr, gpm_lose_yr)):
    ax.text(i - w/2, wv + 2, f'{wv:.0f}', ha='center', fontsize=8)
    ax.text(i + w/2, lv + 2, f'{lv:.0f}', ha='center', fontsize=8)
plt.tight_layout()
plt.show()

```



10 — Tamaño de ventana histórica (freq) y win rate

Se analiza si equipos con **más partidas en su historial** tienen win rates más estables y predecibles.

```

df_valid['freq_bucket'] = pd.cut(df_valid['freq_r'],
    bins=[0, 10, 30, 60, 100, 200, 400],
    labels=['1-10', '11-30', '31-60', '61-100', '101-200', '201+'])

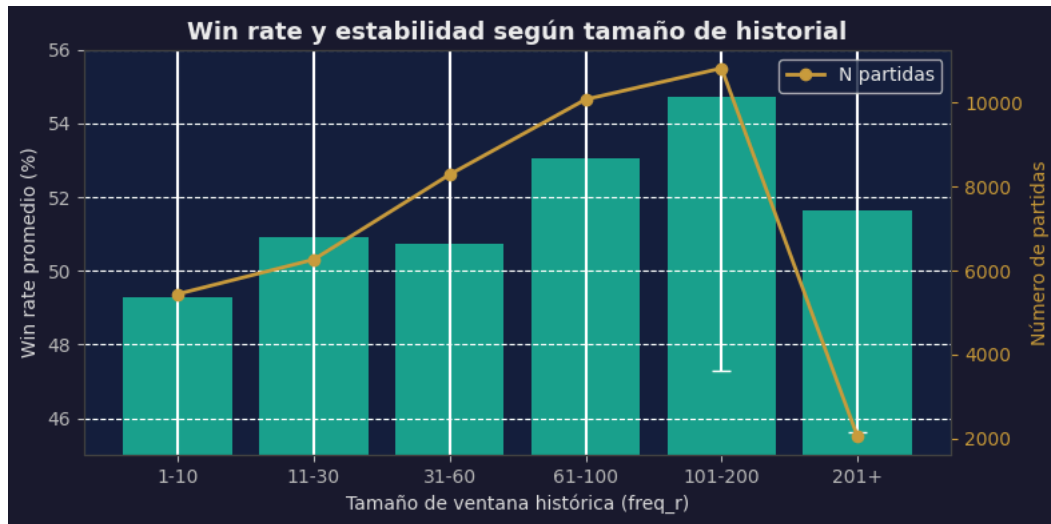
bucket_stats = df_valid.groupby('freq_bucket', observed=True).agg(
    win_pct_mean=('win_pct_r', 'mean'),
    win_pct_std=('win_pct_r', 'std'),
    count=('win_pct_r', 'count')
).reset_index()

fig, ax = plt.subplots(figsize=(8, 4))
bars = ax.bar(bucket_stats['freq_bucket'].astype(str), bucket_stats['win_pct_mean'] * 100,
    color=CYAN, alpha=0.85)
ax.errorbar(range(len(bucket_stats)),
    bucket_stats['win_pct_mean'] * 100,
    yerr=bucket_stats['win_pct_std'] * 100,
    fmt='none', color='white', capsize=5, linewidth=1.5)

ax2 = ax.twinx()
ax2.plot(range(len(bucket_stats)), bucket_stats['count'],
    color=GOLD, marker='o', linewidth=2, label='N partidas')
ax2.set_ylabel('Número de partidas', color=GOLD)
ax2.tick_params(axis='y', colors=GOLD)

ax.axhline(50, color='white', linewidth=0.8, linestyle='--', alpha=0.5)
ax.set_xlabel('Tamaño de ventana histórica (freq_r)')
ax.set_ylabel('Win rate promedio (%)')
ax.set_title('Win rate y estabilidad según tamaño de historial')
ax.set_ylim(45, 56)
ax.grid(axis='y')
ax2.legend(loc='upper right')
plt.tight_layout()
plt.show()

```



11 — Matriz de correlación con radiant_win

Se calculan las correlaciones de las principales estadísticas de Radiant con `radiant_win` para identificar qué features tienen mayor poder predictivo.

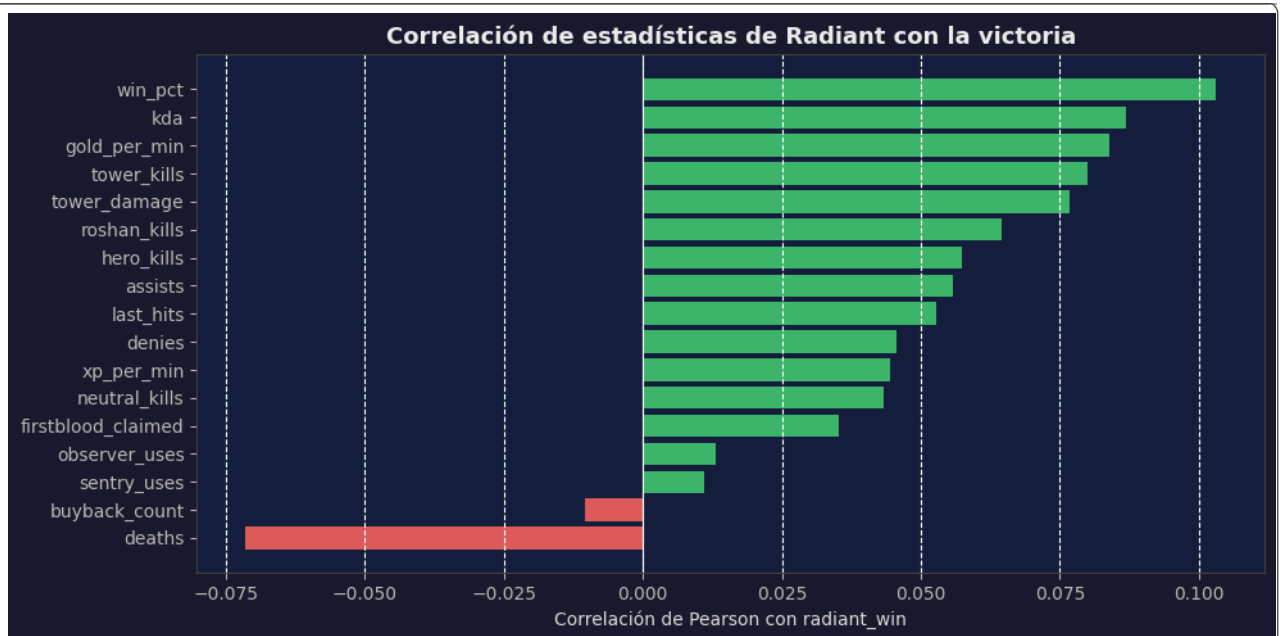
```
stat_cols_r = [
    'win_pct_r', 'kda_avg_r', 'gold_per_min_avg_r', 'xp_per_min_avg_r',
    'hero_kills_avg_r', 'deaths_avg_r', 'assists_avg_r',
    'tower_kills_avg_r', 'tower_damage_avg_r', 'roshan_kills_avg_r',
    'observer_uses_avg_r', 'sentry_uses_avg_r',
    'last_hits_avg_r', 'denies_avg_r', 'neutral_kills_avg_r',
    'firstblood_claimed_avg_r', 'buyback_count_avg_r',
]

corrs = df_valid[stat_cols_r + ['radiant_win']].corr()['radiant_win'].drop('radiant_win')
corrs_sorted = corrs.sort_values(ascending=False)

colors_corr = [GREEN if v > 0 else RED for v in corrs_sorted.values]
labels_corr = [c.replace('_avg_r', '').replace('_r', '') for c in corrs_sorted.index]

fig, ax = plt.subplots(figsize=(10, 5))
ax.barh(labels_corr[::-1], corrs_sorted.values[::-1], color=colors_corr[::-1])
ax.axvline(0, color='white', linewidth=0.8)
ax.set_xlabel('Correlación de Pearson con radiant_win')
ax.set_title('Correlación de estadísticas de Radiant con la victoria')
ax.grid(axis='x')
plt.tight_layout()
plt.show()

print('\nTop 5 correlaciones positivas:')
print(corrs_sorted.head(5).to_string())
print('\nTop 5 correlaciones negativas:')
print(corrs_sorted.tail(5).to_string())
```



Top 5 correlaciones positivas:

win_pct_r	0.103045
kda_avg_r	0.086935
gold_per_min_avg_r	0.083990
tower_kills_avg_r	0.080043
tower_damage_avg_r	0.076841

Top 5 correlaciones negativas:

firstblood_claimed_avg_r	0.035321
observer_uses_avg_r	0.013013
sentry_uses_avg_r	0.011087
buyback_count_avg_r	-0.010357
deaths_avg_r	-0.071419

Resumen de hallazgos